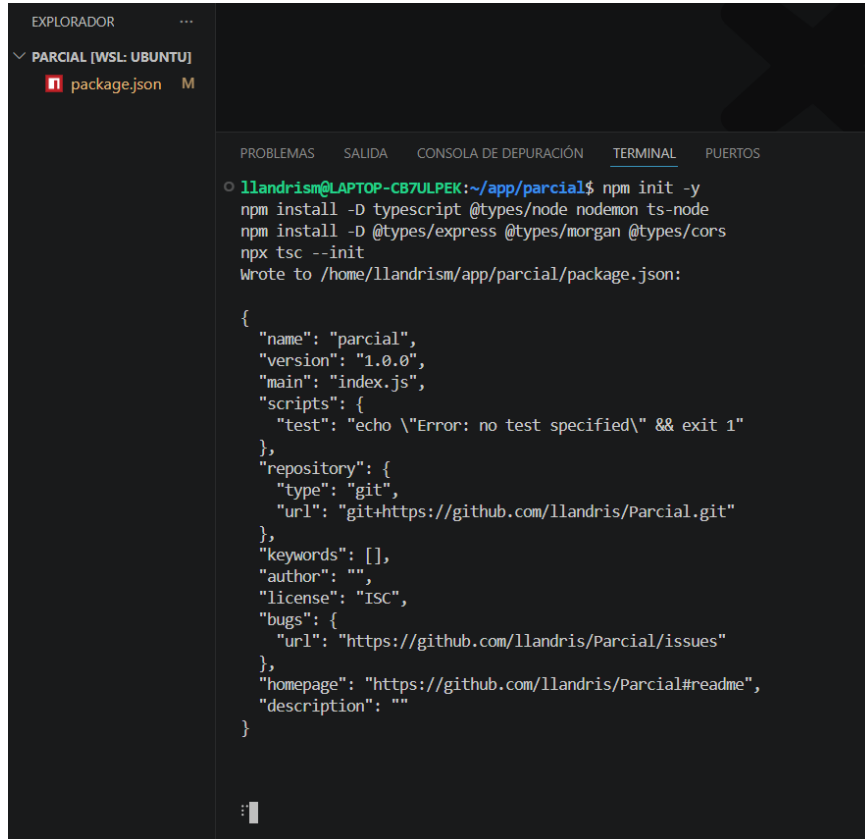


Paso a paso parcial 2

1. instalaciones de Configuración inicial de TypeScript.

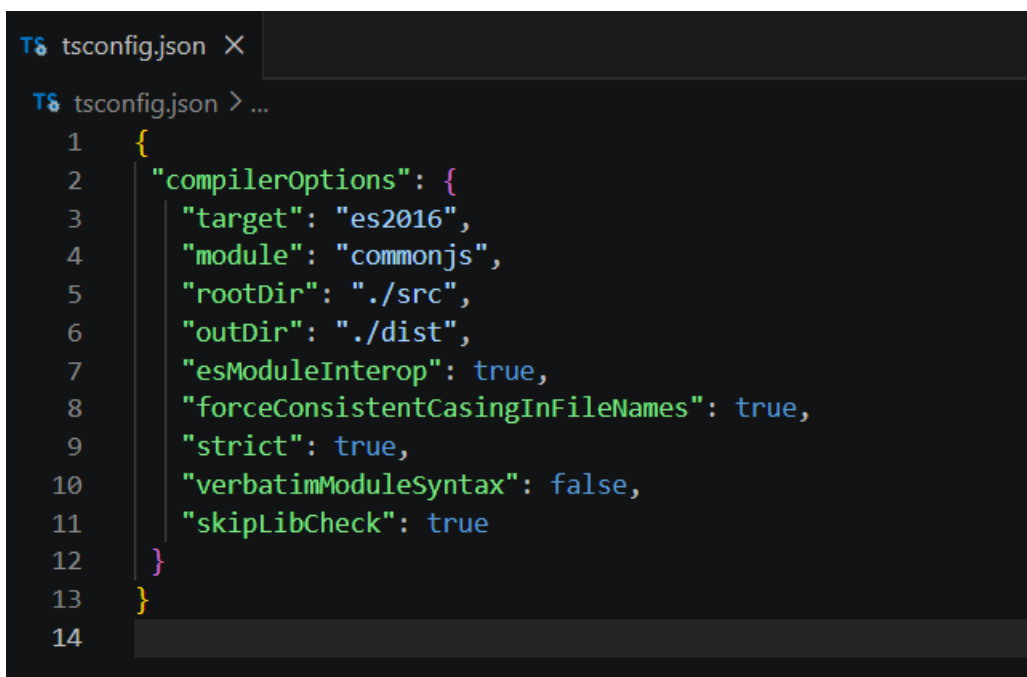


The screenshot shows a VS Code interface with a terminal window open. The terminal displays the following commands and output:

```
llandris@LAPTOP-CB7ULPEK:~/app/parcial$ npm init -y
npm install -D typescript @types/node nodemon ts-node
npm install -D @types/express @types/morgan @types/cors
npx tsc --init
Wrote to /home/llandris/app/parcial/package.json:

{
  "name": "parcial",
  "version": "1.0.0",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "repository": {
    "type": "git",
    "url": "git+https://github.com/llandris/Parcial.git"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "bugs": {
    "url": "https://github.com/llandris/Parcial/issues"
  },
  "homepage": "https://github.com/llandris/Parcial#readme",
  "description": ""
}
```

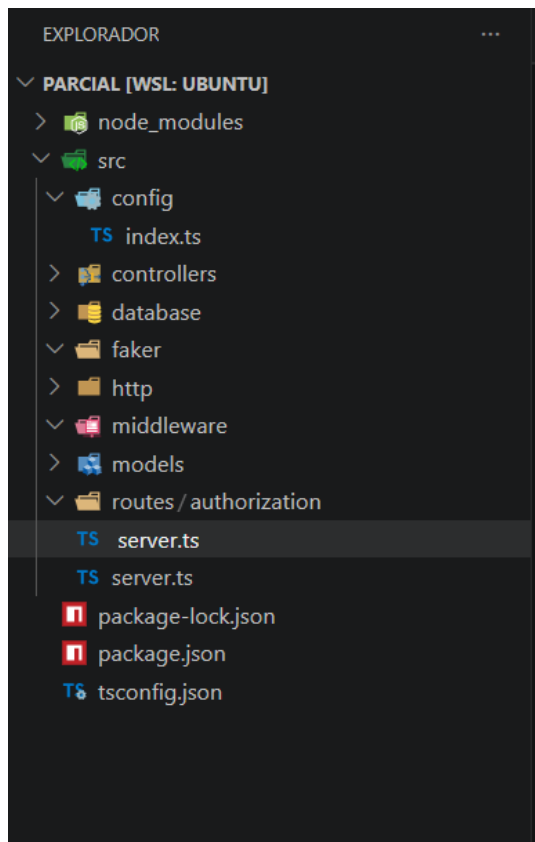
2. cambios en la carpeta tsconfig.json



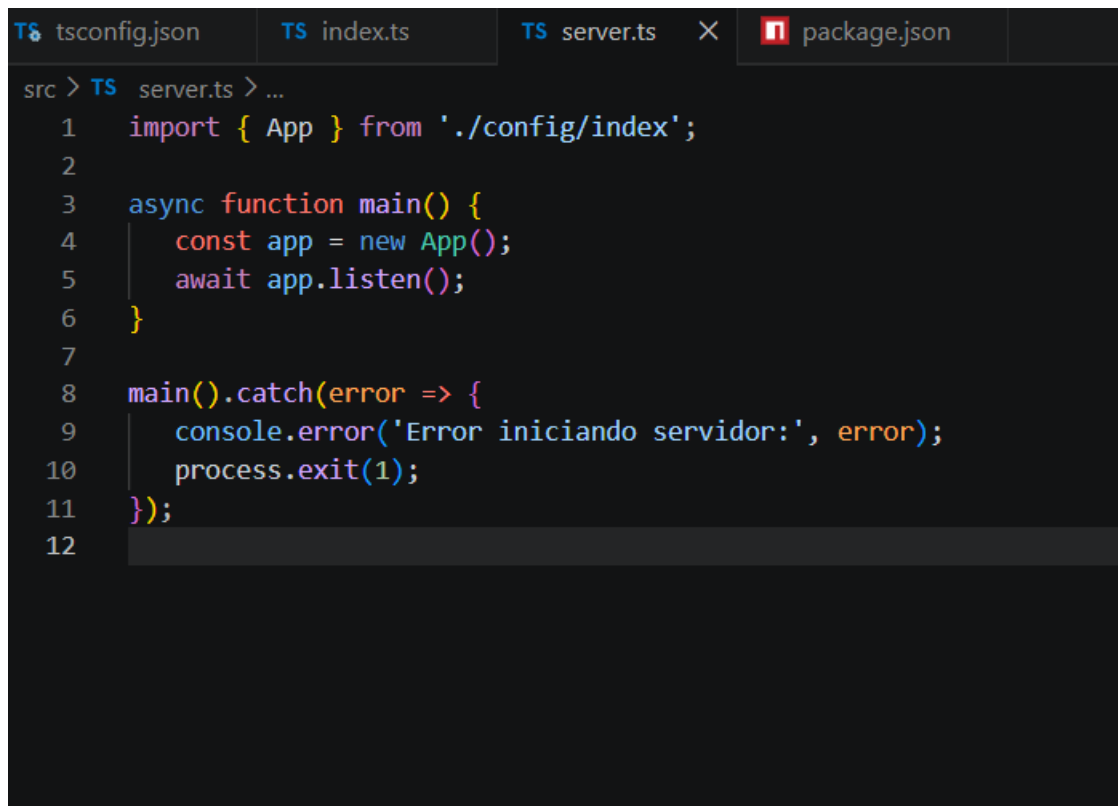
The screenshot shows a VS Code editor with a file named `tsconfig.json` open. The file content is as follows:

```
1  {
2    "compilerOptions": {
3      "target": "es2016",
4      "module": "commonjs",
5      "rootDir": "./src",
6      "outDir": "./dist",
7      "esModuleInterop": true,
8      "forceConsistentCasingInFileNames": true,
9      "strict": true,
10     "verbatimModuleSyntax": false,
11     "skipLibCheck": true
12   }
13 }
14
```

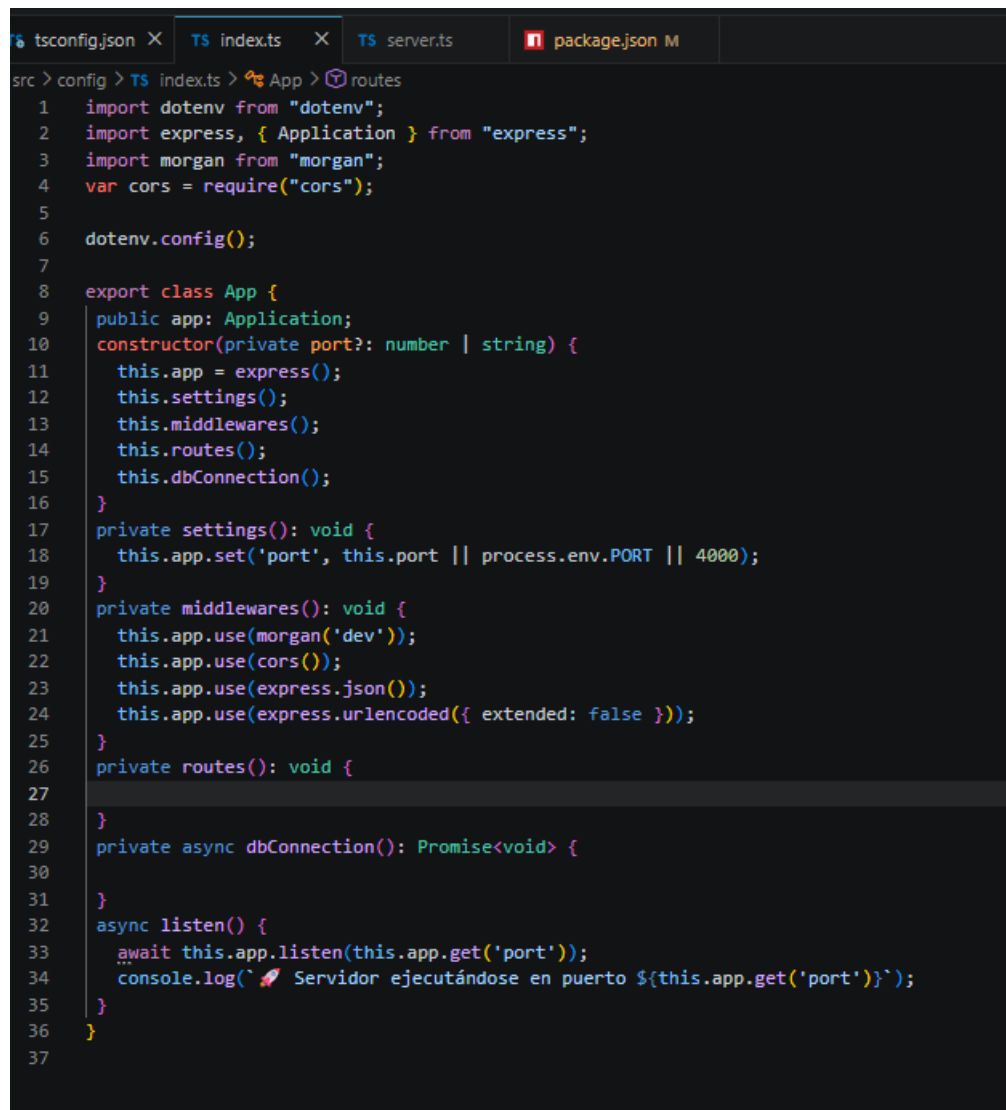
3. creacion de estructura



4. server



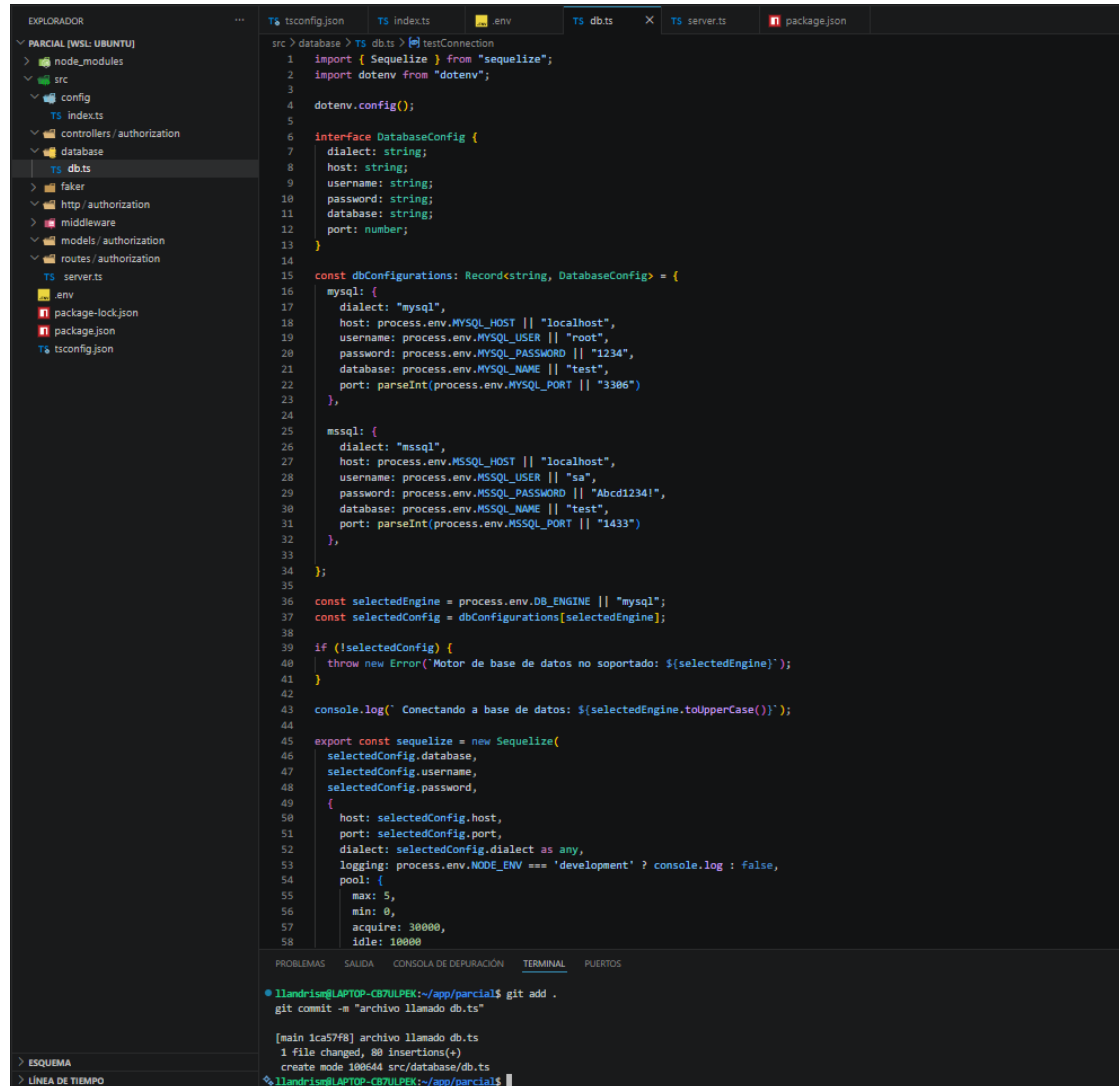
5.. index



The image shows a code editor with four tabs: `tsconfig.json`, `TS index.ts` (active), `TS server.ts`, and `package.json M`. The breadcrumb navigation at the top of the editor area reads: `src > config > TS index.ts > App > routes`. The main editor area displays the following TypeScript code:

```
1  import dotenv from "dotenv";
2  import express, { Application } from "express";
3  import morgan from "morgan";
4  var cors = require("cors");
5
6  dotenv.config();
7
8  export class App {
9    public app: Application;
10   constructor(private port?: number | string) {
11     this.app = express();
12     this.settings();
13     this.middlewares();
14     this.routes();
15     this.dbConnection();
16   }
17   private settings(): void {
18     this.app.set('port', this.port || process.env.PORT || 4000);
19   }
20   private middlewares(): void {
21     this.app.use(morgan('dev'));
22     this.app.use(cors());
23     this.app.use(express.json());
24     this.app.use(express.urlencoded({ extended: false }));
25   }
26   private routes(): void {
27
28   }
29   private async dbConnection(): Promise<void> {
30
31   }
32   async listen() {
33     await this.app.listen(this.app.get('port'));
34     console.log(`🚀 Servidor ejecutándose en puerto ${this.app.get('port')}`);
35   }
36 }
37
```

Base de datos



The image shows a VS Code editor with a file explorer on the left and a code editor in the center. The file explorer shows a project structure with folders like 'src', 'controllers/authorization', 'database', 'faker', 'http/authorization', 'middleware', 'models/authorization', 'routes/authorization', 'server.ts', 'env', 'package-lock.json', 'package.json', and 'tsconfig.json'. The 'database' folder is expanded, showing 'db.ts'. The code editor displays the content of 'db.ts', which is a TypeScript file for database configuration. The code includes imports for 'Sequelize' and 'dotenv', a 'DatabaseConfig' interface, and a 'dbConfigurations' object with 'mysql' and 'mssql' entries. It also includes logic to select a database engine and connect to it using 'sequelize'.

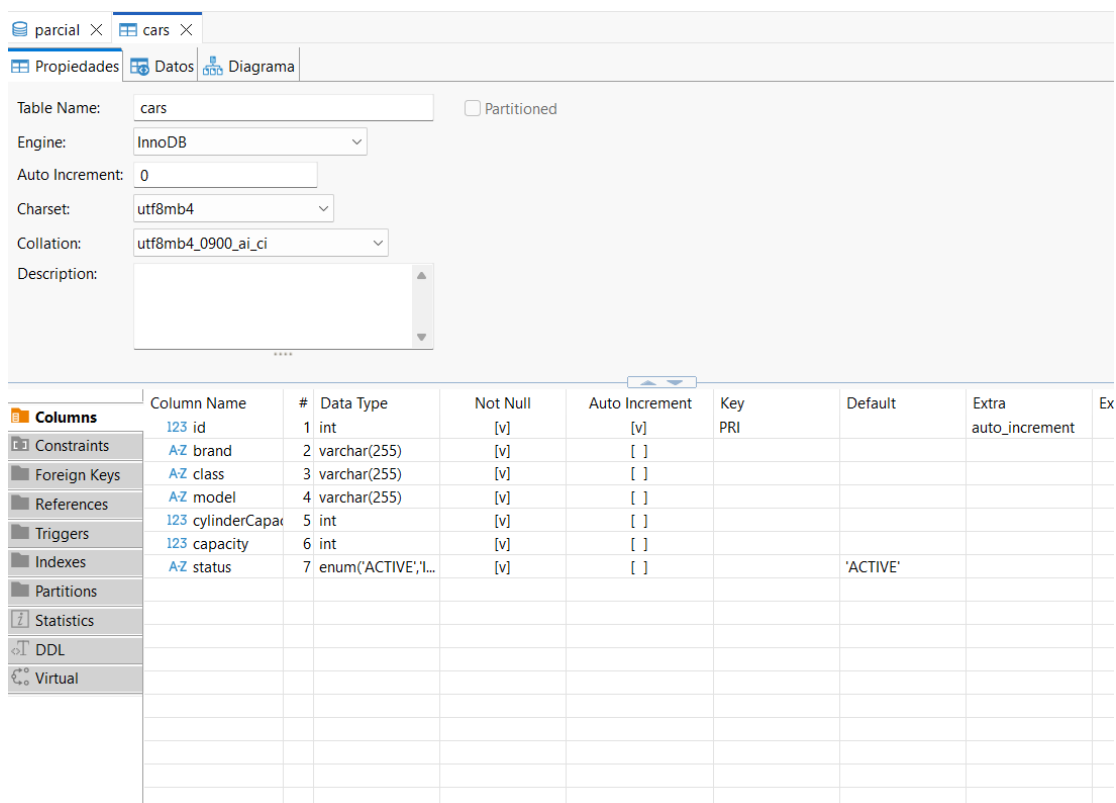
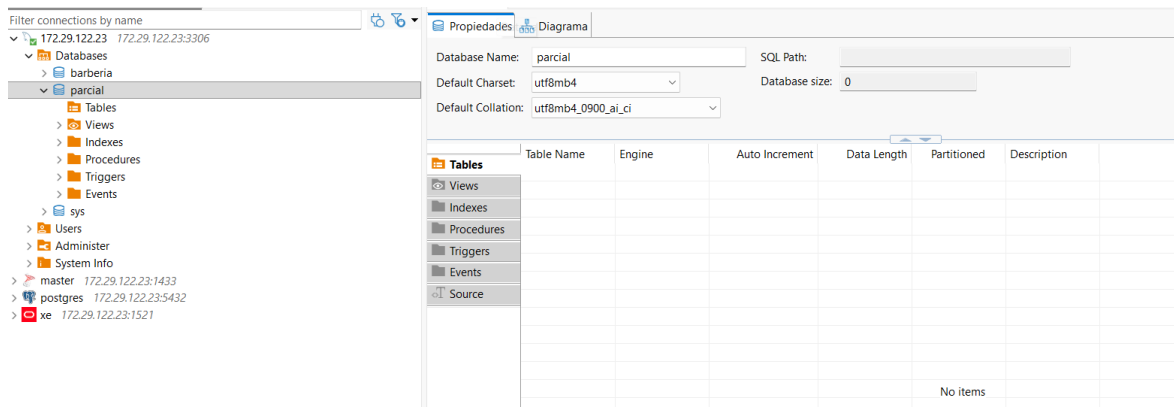
```
src > database > ts db.ts > testConnection
1 import { Sequelize } from "sequelize";
2 import dotenv from "dotenv";
3
4 dotenv.config();
5
6 interface DatabaseConfig {
7   dialect: string;
8   host: string;
9   username: string;
10  password: string;
11  database: string;
12  port: number;
13 }
14
15 const dbConfigurations: Record<string, DatabaseConfig> = {
16   mysql: {
17     dialect: "mysql",
18     host: process.env.MYSQL_HOST || "localhost",
19     username: process.env.MYSQL_USER || "root",
20     password: process.env.MYSQL_PASSWORD || "1234",
21     database: process.env.MYSQL_NAME || "test",
22     port: parseInt(process.env.MYSQL_PORT || "3306")
23   },
24   mssql: {
25     dialect: "mssql",
26     host: process.env.MSSQL_HOST || "localhost",
27     username: process.env.MSSQL_USER || "sa",
28     password: process.env.MSSQL_PASSWORD || "Abcd1234!",
29     database: process.env.MSSQL_NAME || "test",
30     port: parseInt(process.env.MSSQL_PORT || "1433")
31   },
32 };
33
34
35
36 const selectedEngine = process.env.DB_ENGINE || "mysql";
37 const selectedConfig = dbConfigurations[selectedEngine];
38
39 if (!selectedConfig) {
40   throw new Error("Motor de base de datos no soportado: ${selectedEngine}");
41 }
42
43 console.log(` Conectando a base de datos: ${selectedEngine.toUpperCase()} `);
44
45 export const sequelize = new Sequelize(
46   selectedConfig.database,
47   selectedConfig.username,
48   selectedConfig.password,
49   {
50     host: selectedConfig.host,
51     port: selectedConfig.port,
52     dialect: selectedConfig.dialect as any,
53     logging: process.env.NODE_ENV === 'development' ? console.log : false,
54     pool: {
55       max: 5,
56       min: 0,
57       acquire: 30000,
58       idle: 10000
59     }
60   }
61 );
```

The terminal window at the bottom shows the following output:

```
llandris@LAPTOP-C87ULPEK:~/app/parcial$ git add .
git commit -m "archivo llamado db.ts"

[main 1ca57f8] archivo llamado db.ts
1 file changed, 80 insertions(+)
create mode 100644 src/database/db.ts
llandris@LAPTOP-C87ULPEK:~/app/parcial$
```

Creacion de base de datos física mysql



parcial

cars

tutions

Propiedades

Datos

Diagrama

Table Name:

tutions

☐ Partitioned

Engine:

InnoDB

Auto Increment:

0

Charset:

utf8mb4

Collation:

utf8mb4_0900_ai_ci

Description:

	Column Name	#	Data Type	Not Null	Auto Increment	Key	Default	Extra	Expression	Comment
	<u>123</u> id	1	int	[v]	[v]	PRI		auto_increment		
	<u>123</u> car_id	2	int	[v]	[]	MUL				
	<u>AZ</u> plate_number	3	varchar(255)	[v]	[]					
	<u>123</u> registration_date	4	datetime	[v]	[]					
	<u>123</u> expiration_date	5	datetime	[v]	[]					
	<u>AZ</u> status	6	enum('ACTIVE',...)	[v]	[]		'ACTIVE'			

Columns

Constraints

Foreign Keys

References

Triggers

Indexes

Partitions

Statistics

DDL

Virtual